

Quantitative Finance with QuantLib

From Basic Swaps to Callable Cross-Currency Derivatives
A Comprehensive Technical Journey

Quantitative Analytics Team

TGIR QuantLib Tools Project

August 2026

Part I: Foundations

- Financial Derivatives Basics
- Time Value of Money
- Yield Curves
- QuantLib Introduction

Part II: Interest Rate Products

- Interest Rate Swaps
- SOFR and OIS Markets
- European Swaptions
- Bermudan Swaptions

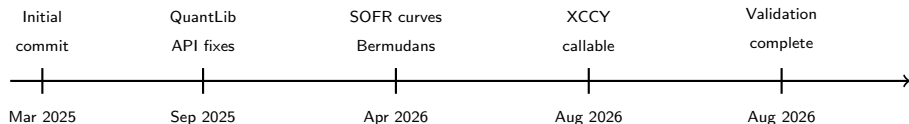
Part III: Cross-Currency & Exotics

- FX Derivatives Basics
- Cross-Currency Swaps
- Callable XCCY Structures
- Three-Factor Models

Part IV: Implementation

- Monte Carlo Methods
- LSM Algorithm
- QuantLib Architecture
- C++ vs Python

Project Timeline: A Chronological Journey



Key Milestones:

- **March 2025:** Initial GPT-4.5 prototype with basic swap pricing
- **September 2025:** QuantLib API compatibility updates
- **April 2026:** SOFR curve bootstrap, Bermudan swaption engine
- **August 2026:** Three-factor callable XCCY with LSM Monte Carlo

What is a Financial Derivative?

Definition

A **derivative** is a financial contract whose value is *derived* from an underlying asset, rate, or index.

Common Underlying Assets:

- **Interest Rates:** SOFR, ESTR, Treasury yields
- **Equities:** Stock prices, indices (S&P 500)
- **Foreign Exchange:** Currency pairs (EUR/USD)
- **Commodities:** Oil, gold, agricultural products
- **Credit:** Default probabilities, credit spreads

Purpose:

- Hedging existing exposures
- Speculation on price movements
- Arbitrage opportunities
- Access to markets/payoffs otherwise unavailable

Types of Derivatives

Linear Derivatives

- Forwards
- Futures
- Swaps

Payoff is linear in the underlying.

Non-Linear Derivatives

- Options (calls, puts)
- Caps/Floors
- Swaptions

Payoff has optionality (asymmetric).

Vanilla vs Exotic

- **Vanilla:** Standard, liquid, exchange-traded
- **Exotic:** Complex payoffs, OTC, bespoke

Exotic Examples:

- Barrier options
- Asian options (averaging)
- Bermudan exercise
- Callable/puttable structures
- Cross-currency hybrids

Time Value of Money: Foundation of Pricing

Present Value

\$1 today is worth more than \$1 in the future due to:

- Opportunity cost (investment returns)
- Inflation
- Risk/uncertainty

Discount Factor:

$$D(0, T) = \frac{1}{(1 + r)^T} \quad (\text{discrete})$$

$$D(0, T) = e^{-rT} \quad (\text{continuous})$$

Present Value of Future Cashflow:

$$PV = \sum_{i=1}^n CF_i \times D(0, t_i)$$

All derivative pricing reduces to computing expected discounted cashflows.

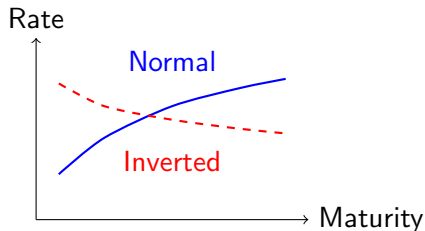
Yield Curves: The Term Structure of Interest Rates

What is a Yield Curve?

A yield curve shows how interest rates vary with maturity (term).

Typical Shapes:

- **Normal:** Upward sloping (short < long)
- **Inverted:** Downward sloping (short > long)
- **Flat:** Similar rates across maturities
- **Humped:** Peak in middle maturities



Key Concept: Derivatives are priced using *risk-free* discount curves, typically derived from government bonds or overnight index swaps (OIS).

From LIBOR to SOFR: The Rate Reform

The LIBOR Scandal and Reform

LIBOR (London Interbank Offered Rate) was phased out due to manipulation scandals and lack of underlying transactions.

New Risk-Free Rates (RFRs):

- **SOFR** (Secured Overnight Financing Rate) - USD
- **ESTR/€STR** (Euro Short-Term Rate) - EUR
- **SONIA** (Sterling Overnight Index Average) - GBP
- **TONA** (Tokyo Overnight Average Rate) - JPY

Key Differences from LIBOR:

- Overnight rates (not term rates)
- Transaction-based (not survey-based)
- Near risk-free (secured or central bank)
- Compounded in arrears for term periods

Overnight Index Swaps (OIS)

Definition

An OIS is a swap where one party pays a fixed rate and receives the compounded overnight rate over the swap period.

Structure:

- Fixed Leg: Pay fixed rate K on notional N
- Floating Leg: Receive daily compounded SOFR

SOFR Compounding:

$$\text{Floating Payment} = N \times \left[\prod_{i=1}^n \left(1 + r_i \times \frac{d_i}{360} \right) - 1 \right]$$

Where r_i is the daily SOFR rate and d_i is the day count.

Importance: OIS rates are used to construct the *discount curve* for derivative pricing.

Introduction to QuantLib

What is QuantLib?

- Open-source C++ library for quantitative finance
- Started in 2000, mature and widely used
- Covers interest rates, equity, FX, credit
- Python bindings via SWIG (QuantLib-Python)

Why QuantLib?

- Industry-standard implementations
- Extensive model library
- Active community and documentation
- Free and open source

<https://www.quantlib.org>

Core Components:

- Term structures
- Instruments
- Pricing engines
- Calendars
- Day counters
- Optimization
- Monte Carlo

QuantLib: Basic Setup in Python

```
import QuantLib as ql

# Set the evaluation date (critical for all pricing!)
today = ql.Date(14, 8, 2026)
ql.Settings.instance().evaluationDate = today

# Basic date arithmetic
one_year_later = today + ql.Period(1, ql.Years)
print(f"Today: {today}")
print(f"One year later: {one_year_later}")

# Calendars handle business day conventions
calendar = ql.UnitedStates(ql.UnitedStates.Settlement)
spot_date = calendar.advance(today, 2, ql.Days)
print(f"Spot date (T+2): {spot_date}")
```

Key Concept: Always set evaluationDate before any pricing!

QuantLib: Day Count Conventions

```
import QuantLib as ql

# Common day count conventions
act360 = ql.Actual360()           # Money markets
act365 = ql.Actual365Fixed()      # Some bonds
thirty360 = ql.Thirty360(ql.Thirty360.BondBasis)  # US bonds

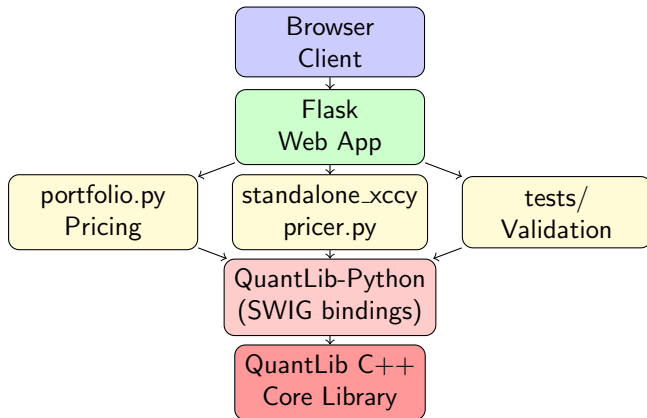
# Calculate year fractions
start = ql.Date(1, 1, 2026)
end = ql.Date(1, 7, 2026)

print(f"ACT/360: {act360.yearFraction(start, end):.6f}")
print(f"ACT/365: {act365.yearFraction(start, end):.6f}")
print(f"30/360: {thirty360.yearFraction(start, end):.6f}")
```

Output:

- ACT/360: 0.502778 (181/360)
- ACT/365: 0.495890 (181/365)
- 30/360: 0.500000 (180/360)

Project Architecture Overview



Development Environment Setup

Prerequisites

- Python 3.10+ (we use 3.13)
- QuantLib-Python package
- Virtual environment

Installation Steps:

- 1 Create virtual environment:
 - `python3 -m venv .venv`
 - `source .venv/bin/activate`
- 2 Install dependencies:
 - `pip install QuantLib-Python numpy pandas`
 - `pip install flask pytest`
- 3 Verify installation:
 - `python -c "import QuantLib; print(QuantLib.__version__)"`

Git Commit History: Initial Development (March 2025)

Commit: c78135e - Initial commit (March 15, 2025)

- Empty repository setup
- License file (Apache 2.0)

Commit: 8e17f9f - GPT-4.5 iteration (March 15, 2025)

- First prototype generated with AI assistance
- Basic swap and swaption structures
- No test runs yet

Key Learning:

- AI-assisted code generation for quantitative finance
- Starting point for iterative refinement
- Importance of validation and testing

Git Commit History: API Stabilization (September 2025)

Commit: 6bea9e2 - Fix QuantLib API updates (Sep 3, 2025)

- `ql.UnitedStates.Settlement` enum changes
- Swaption setup modifications
- Compatibility with QuantLib updates

Commit: 0bf36bb - Add detailed README

- Documentation of project structure
- Installation instructions
- Usage examples

Key Learning:

- QuantLib is actively developed; APIs change
- Always check version compatibility
- Good documentation is essential

Part II

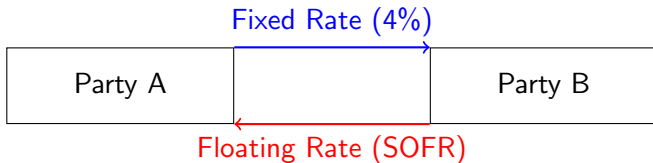
Interest Rate Products

Swaps, Curves, and Swaptions

Interest Rate Swap: The Building Block

Definition

An **interest rate swap** is an agreement to exchange interest payments on a notional principal, typically fixed-for-floating.

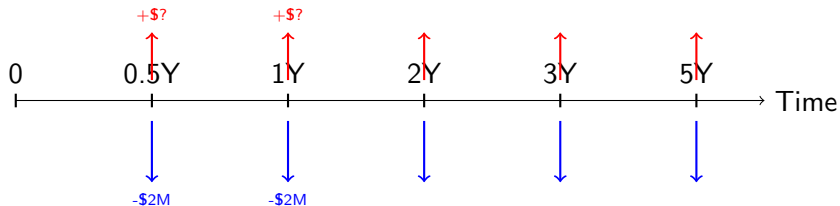


Key Terms:

- **Notional:** Reference amount (e.g., \$100M)
- **Fixed Rate:** Predetermined rate (e.g., 4%)
- **Floating Rate:** Varies with index (SOFR)
- **Tenor:** Length of swap (e.g., 5 years)
- **Payment Frequency:** Quarterly, semi-annual, annual

Swap Cash Flows Example

Example: 5-year USD swap, \$100M notional, 4% fixed vs SOFR, semi-annual



Fixed payment = $\frac{\$100M \times 4\%}{2} = \$2M$ per period

Floating payment = $\$100M \times \text{compounded SOFR} \times \text{accrual}$

Swap Valuation: NPV Calculation

Net Present Value (NPV) of a Swap:

$$NPV_{swap} = NPV_{floating} - NPV_{fixed}$$

Fixed Leg PV:

$$PV_{fixed} = \sum_{i=1}^n N \times K \times \delta_i \times D(0, t_i)$$

Floating Leg PV:

$$PV_{floating} = \sum_{i=1}^n N \times F_i \times \delta_i \times D(0, t_i)$$

Where:

- N = Notional
- K = Fixed rate
- F_i = Forward rate for period i
- δ_i = Day count fraction
- $D(0, t_i)$ = Discount factor to payment date

Par Swap Rate

Definition

The **par swap rate** is the fixed rate K^* that makes $NPV_{swap} = 0$.

Setting $NPV = 0$:

$$K^* = \frac{\sum_{i=1}^n F_i \times \delta_i \times D(0, t_i)}{\sum_{i=1}^n \delta_i \times D(0, t_i)}$$

The denominator is called the **annuity factor** or DV01-like quantity.

Par Rates from Market:

- Liquid swap rates are quoted as par rates
- Used to bootstrap the yield curve
- Interpolation fills gaps between quoted maturities

Building a SOFR Curve in QuantLib

```
import QuantLib as ql

today = ql.Date(14, 8, 2026)
ql.Settings.instance().evaluationDate = today

# SOFR index
sofr = ql.Sofr()

# Market quotes (OIS rates)
quotes = [
    ("1W", 0.0410), ("1M", 0.0408), ("3M", 0.0404),
    ("6M", 0.0398), ("1Y", 0.0390), ("2Y", 0.0378),
    ("5Y", 0.0371), ("10Y", 0.0385), ("30Y", 0.0395)
]

# Build OIS helpers
helpers = []
for tenor, rate in quotes:
    helpers.append(ql.OISRateHelper(
        2, ql.Period(tenor),
        ql.QuoteHandle(ql.SimpleQuote(rate)),
        sofr
    ))
```

Using the SOFR Curve

```
# Wrap curve in a handle for use in instruments
curve_handle = ql.YieldTermStructureHandle(curve)

# Get discount factors
print("Discount Factors:")
for years in [1, 2, 5, 10, 30]:
    date = today + ql.Period(years, ql.Years)
    df = curve_handle.discount(date)
    print(f"    {years}Y: {df:.6f}")

# Get zero rates (continuous)
print("\nZero Rates:")
for years in [1, 2, 5, 10, 30]:
    date = today + ql.Period(years, ql.Years)
    zero = curve_handle.zeroRate(date, ql.Actual365Fixed(),
                                  ql.Continuous).rate()
    print(f"    {years}Y: {zero*100:.3f}%")
```

Curve Bootstrap: How It Works

Bootstrapping

Starting from the shortest maturity, sequentially solve for discount factors that reprice each market instrument.

Algorithm:

- 1 Start with overnight rate (1D deposit)
- 2 For each successive OIS quote:
 - Use already-known discount factors
 - Solve for the new discount factor
 - Interpolate between nodes
- 3 Result: Full discount curve

Interpolation Methods:

- **LogLinear:** Interpolate log of discount factors
- **Cubic:** Smooth forwards
- **NaturalCubic:** Zero second derivative at ends

Curve Calibration: Repricing Test

Quality Check: Calibrated curve should exactly reprice input instruments.

Tenor	Market Rate	Model Rate	Error (bp)
1W	4.100%	4.100%	0.000
1M	4.080%	4.080%	0.000
3M	4.040%	4.040%	0.000
6M	3.980%	3.980%	0.000
1Y	3.900%	3.900%	0.000
2Y	3.780%	3.780%	0.000
5Y	3.710%	3.710%	0.000
10Y	3.850%	3.850%	0.000

Tolerance: Errors should be $< 0.01\text{bp}$ for production use.

Pricing an Interest Rate Swap in QuantLib

```
# Create SOFR index linked to curve
sofr_index = ql.Sofr(curve_handle)

# Create swap schedule
calendar = ql.UnitedStates(ql.UnitedStates.Settlement)
start = calendar.advance(today, 2, ql.Days)
maturity = start + ql.Period(5, ql.Years)

schedule = ql.Schedule(start, maturity,
                        ql.Period(ql.Semiannual), calendar,
                        ql.ModifiedFollowing, ql.ModifiedFollowing,
                        ql.DateGeneration.Backward, False)

# Create OIS swap
notional = 100_000_000
fixed_rate = 0.04
swap = ql.OvernightIndexedSwap(
    ql.Swap.Payer, notional, schedule,
    fixed_rate, ql.Actual360(), sofr_index)

# Price with discounting engine
swap.setPricingEngine(ql.DiscountingSwapEngine(curve_handle))
print(f"NPV: ${swap.NPV():,.2f}")
```

Swap Risk Measures: DV01 and PV01

DV01 (Dollar Value of 01)

Change in swap value for a 1 basis point parallel shift in rates.

$$DV01 = \frac{V(r + 0.01\%) - V(r - 0.01\%)}{2}$$

For a Receiver Swap:

- Receive fixed, pay floating
- Gains when rates fall (positive DV01)
- $DV01 \approx \text{Notional} \times \text{Duration} \times 0.0001$

Typical 5Y Swap DV01:

- \$100M notional
- Duration ≈ 4.5 years
- $DV01 \approx \$45,000$ per bp

European Swaption: Option on a Swap

Definition

A **swaption** is an option to enter into an interest rate swap at a predetermined fixed rate.

Key Parameters:

- **Expiry:** When the option can be exercised
- **Underlying Tenor:** Length of the swap if exercised
- **Strike:** Fixed rate of the underlying swap
- **Type:** Payer (right to pay fixed) or Receiver (right to receive fixed)

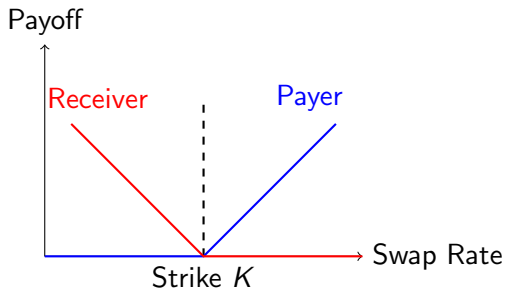
Notation: “2Y x 5Y swaption” means:

- Option expires in 2 years
- If exercised, enter a 5-year swap
- Total exposure: 7 years from today

Swaption Payoff

At Expiry (European):

- **Payer Swaption:** $\max(0, \text{Swap NPV at expiry})$
- **Receiver Swaption:** $\max(0, -\text{Swap NPV at expiry})$



Intrinsic Value: $\text{Annuity} \times \max(0, \text{Forward Rate} - K)$

Swaption Volatility: Normal vs Lognormal

Two Common Volatility Conventions: Normal (Bachelier) Volatility

- Rates can go negative
- Quoted in bp (e.g., 75bp)
- $dS_t = \sigma dW_t$
- Preferred post-2008

Lognormal (Black) Volatility

- Rates must be positive
- Quoted in % (e.g., 20%)
- $dS_t = \sigma S_t dW_t$
- Historical standard

Conversion (ATM approximation):

$$\sigma_{normal} \approx \sigma_{Black} \times K$$

Example: If $K = 4\%$ and $\sigma_{Black} = 20\%$, then $\sigma_{normal} \approx 80$ bp.

Swaption Volatility Surface

Expiry	Underlying Swap Tenor				
	1Y	2Y	5Y	10Y	30Y
1M	45	52	68	75	72
3M	48	55	70	77	74
1Y	55	62	76	82	78
2Y	60	68	80	85	80
5Y	65	72	82	86	82
10Y	68	75	84	88	84

Normal volatilities in basis points (bp)

Observations:

- Vol generally increases with expiry (time decay)
- Longer tenors often have higher vols
- Surface shape varies with market conditions

Pricing a European Swaption in QuantLib

```
# Build the underlying swap
expiry = 2   # years
tenor = 5    # years
strike = 0.04

# Swaption schedule
expiry_date = today + ql.Period(expiry, ql.Years)
swap_start = expiry_date
swap_maturity = swap_start + ql.Period(tenor, ql.Years)

# Create swaption
exercise = ql.EuropeanExercise(expiry_date)
underlying_swap = make_underlying_swap() # setup as before

swaption = ql.Swaption(underlying_swap, exercise)

# Normal volatility quote
vol = 0.0080 # 80 bp
vol_handle = ql.QuoteHandle(ql.SimpleQuote(vol))
engine = ql.BachelierSwaptionEngine(
    curve_handle, vol_handle, ql.Actual365Fixed())

swaption.setPricingEngine(engine)
```


Hull-White Model: Short Rate Dynamics

Hull-White One-Factor Model

$$dr_t = (\theta(t) - a \cdot r_t) dt + \sigma dW_t$$

Parameters:

- r_t = Short rate at time t
- $\theta(t)$ = Time-dependent drift (fits initial curve)
- a = Mean reversion speed
- σ = Volatility
- dW_t = Brownian motion increment

Key Properties:

- **Mean Reversion:** Rates pulled toward long-run level
- **Analytical Tractability:** Closed-form bond prices
- **Calibration:** Fit $\theta(t)$ to today's curve, σ to vol surface

Hull-White Model: Calibration

Calibration Process:

- 1 Fix mean reversion a (often 3% for rates)
- 2 $\theta(t)$ is determined by no-arbitrage (fits curve)
- 3 Calibrate σ to match swaption prices

Swaption Calibration:

- Choose set of calibration instruments (co-terminal swaptions)
- Minimize pricing error across instruments
- Trade-off: One-factor can't fit entire surface

Co-Terminal Swaptions:

- All expire into swaps ending at same date
- Example for 10Y maturity: 2Yx8Y, 3Yx7Y, 4Yx6Y, ..., 9Yx1Y
- Natural for Bermudan calibration

Hull-White Calibration in QuantLib

```
# Create Hull-White model
mean_reversion = 0.03
initial_sigma = 0.01
model = ql.HullWhite(curve_handle, mean_reversion, initial_sigma)

# Create calibration helpers (co-terminal swaptions)
helpers = []
for expiry, tenor in [("2Y", "8Y"), ("3Y", "7Y"), ("5Y", "5Y")]:
    vol_bp = 80.0 # from vol surface
    helper = ql.SwaptionHelper(
        ql.Period(expiry), ql.Period(tenor),
        ql.QuoteHandle(ql.SimpleQuote(vol_bp * 1e-4)),
        sofr_index, ql.Period("1Y"), ql.Actual360(),
        ql.Actual360(), curve_handle,
        ql.BlackCalibrationHelper.RelativePriceError,
        ql.nullDouble(), 1.0, ql.Normal)
    helper.setPricingEngine(ql.JamshidianSwaptionEngine(model))
    helpers.append(helper)

# Calibrate
model.calibrate(helpers, ql.LevenbergMarquardt(),
    ql.EndCriteria(500, 100, 1e-10, 1e-10, 1e-10))
```

Bermudan Swaption: Multiple Exercise Dates

Definition

A **Bermudan swaption** can be exercised on multiple predetermined dates (unlike European which has only one).

Example: 10Y NC2 (10-year, 2-year non-call)

- Total maturity: 10 years
- Non-call period: First 2 years
- Exercise dates: Years 2, 3, 4, 5, 6, 7, 8, 9
- Each exercise enters remaining swap



Bermudan Pricing: Dynamic Programming

At Each Exercise Date (Backward Recursion):

$$V(t) = \max \left(\underbrace{\text{Exercise Value}}_{\text{Enter swap}}, \underbrace{\text{Continuation Value}}_{\text{Wait}} \right)$$

Numerical Methods:

- **Tree Methods:** Discretize short rate on lattice
- **Finite Difference:** Solve PDE on grid
- **Monte Carlo + LSM:** For high dimensions

Tree Advantages:

- Efficient for one-factor models
- Naturally handles early exercise
- QuantLib: TreeSwaptionEngine

Pricing a Bermudan Swaption

```
# Create underlying swap (10Y maturity)
underlying = make_swap()

# Bermudan exercise: annual from year 2 to 9
exercise_dates = [today + ql.Period(y, ql.Years)
                  for y in range(2, 10)]
exercise = ql.BermudanExercise(exercise_dates)

# Create Bermudan swaption
bermudan = ql.Swaption(underlying, exercise)

# Calibrated Hull-White model
model = ql.HullWhite(curve_handle, a, sigma)

# Tree engine (timesteps per year)
engine = ql.TreeSwaptionEngine(model, 50)
bermudan.setPricingEngine(engine)

print(f"Bermudan NPV: ${bermudan.NPV():,.2f}")
```

Bermudan vs European Value

Key Relationship:

$$V_{Bermudan} \geq V_{European}$$

The additional exercise rights always add value (or zero if never optimal).

Exercise Premium:

$$\text{Premium} = V_{Bermudan} - V_{European}$$

Typical Drivers of Early Exercise:

- High coupon swap (positive carry)
- Steep yield curve
- Time value of money
- Funding considerations

G2++ Model

$$r_t = x_t + y_t + \phi(t)$$

$$dx_t = -a \cdot x_t dt + \sigma dW_t^x$$

$$dy_t = -b \cdot y_t dt + \eta dW_t^y$$

$$dW_t^x \cdot dW_t^y = \rho dt$$

Advantages over Hull-White 1F:

- Better fit to volatility surface
- Non-parallel curve movements
- Richer correlation structure

Trade-offs:

- More parameters to calibrate
- Slower computation
- Still analytical for Europeans

Project: Rates Workstation

Git Commit: April 2026

31df798 - Add curve analytics and calibrated Bermudan model

Features Implemented:

- Full SOFR curve bootstrap (22 points)
- ATM swaption volatility matrix
- Hull-White and G2++ calibration
- Bermudan swaption pricing grid
- Interactive web interface

Portfolio of Trades:

- 1 Vanilla Swap (100M notional)
- 2 European Swaption (ATM)
- 3 Bermudan Swaption 1 (10Y NC2)
- 4 Bermudan Swaption 2 (5Y NC1)
- 5 Equity Cliquet

Part III

Equity and FX Derivatives

Cliquets, FX Forwards, and Cross-Currency Basics

Equity Options: Black-Scholes Foundation

Black-Scholes Model

$$dS_t = (r - q)S_t dt + \sigma S_t dW_t$$

Parameters:

- S_t = Stock price
- r = Risk-free rate
- q = Dividend yield
- σ = Volatility

Call Option Price:

$$C = S_0 e^{-qT} N(d_1) - K e^{-rT} N(d_2)$$

Where:

$$d_1 = \frac{\ln(S_0/K) + (r - q + \sigma^2/2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}$$

Cliquet Options: Reset Features

Definition

A **cliquet** (or ratchet) option has its strike reset periodically, typically to the spot price at each reset date.

Structure:

- Series of forward-starting options
- Each period: $\max(0, S_{end}/S_{start} - 1)$ (for calls)
- Total payoff: Sum of periodic returns
- Often includes caps/floors on each period

Example: Monthly Cliquet on SPX

- 1-year maturity, monthly resets
- Each month: Pay $\max(0, R_i - 0)$ where $R_i = S_i/S_{i-1} - 1$
- Total payoff: $\sum_{i=1}^{12} \max(0, R_i)$

Cliquet Pricing in QuantLib

```
import QuantLib as ql

today = ql.Date(5, 4, 2026)
ql.Settings.instance().evaluationDate = today

# Market data
spot = 5000.0
vol = 0.20
rate = 0.04
div_yield = 0.015

# Build process
process = ql.BlackScholesMertonProcess(
    ql.QuoteHandle(ql.SimpleQuote(spot)),
    ql.YieldTermStructureHandle(ql.FlatForward(today, div_yield, ql.Actual365Fixed(
    ))),
    ql.YieldTermStructureHandle(ql.FlatForward(today, rate, ql.Actual365Fixed()))),
    ql.BlackVolTermStructureHandle(
        ql.BlackConstantVol(today, ql.NullCalendar(), vol, ql.Actual365Fixed()))))

# Reset dates (quarterly)
reset_dates = [today + ql.Period(i, ql.Months) for i in [0, 3, 6, 9]]
maturity = today + ql.Period(1, ql.Years)
```

FX Quote Conventions:

- $\text{EUR/USD} = 1.10$ means $1 \text{ EUR} = 1.10 \text{ USD}$
- “USD per EUR” or “EUR in USD terms”
- **Base currency:** EUR (first)
- **Quote currency:** USD (second)

Common Derivatives:

- **FX Forward:** Lock in exchange rate for future date
- **FX Option:** Right to exchange at strike rate
- **FX Swap:** Exchange currencies now, reverse later
- **Cross-Currency Swap:** Exchange rate exposure + interest

FX Forward Pricing

Covered Interest Parity

$$F = S \times \frac{D_{domestic}(T)}{D_{foreign}(T)}$$

Where:

- F = Forward exchange rate
- S = Spot exchange rate
- $D_{domestic}(T)$ = Domestic discount factor
- $D_{foreign}(T)$ = Foreign discount factor

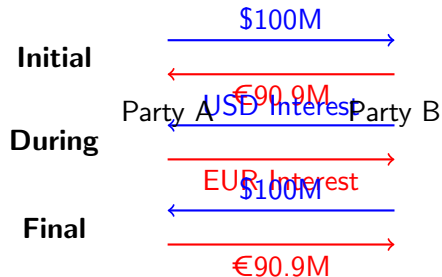
Example (EUR/USD):

- Spot: 1.10
- USD 1Y rate: 4%, EUR 1Y rate: 2.5%
- Forward: $1.10 \times \frac{e^{-0.04}}{e^{-0.025}} = 1.10 \times e^{-0.015} \approx 1.084$

Cross-Currency Swap: Definition

Definition

A **cross-currency swap** exchanges principal and interest payments in two different currencies.



Cross-Currency Swap: Cash Flow Structure

Example: 10Y EUR/USD Cross-Currency Swap

- **Initial Exchange:**

- Party A pays \$100M, receives €90.9M
- Exchange at initial spot rate

- **Periodic Payments:**

- Party A pays: 4% fixed on \$100M (semi-annual)
- Party A receives: ESTR compounded on €90.9M (quarterly)

- **Final Exchange:**

- Party A receives \$100M, pays €90.9M
- Re-exchange at same rate

Key Risk: FX exposure on notional exchanges!

Cross-Currency Basis

Definition

The **cross-currency basis** is the spread added to one leg of a XCCY swap to make it trade at par.

Why does basis exist?

- Supply/demand imbalances for USD funding
- Credit risk differences
- Regulatory capital requirements
- Central bank policy divergence

Market Observation:

- Basis is typically negative for EUR/USD
- Meaning: Pay ESTR *minus* basis to receive USD flat
- Reflects USD funding premium

Callable Cross-Currency Swap: The Challenge

Structure

A **callable cross-currency swap** allows one party to terminate the swap early on predetermined dates.

Complexity:

- **Three Stochastic Factors:**
 - USD short rate
 - EUR short rate
 - EUR/USD exchange rate
- **Early Exercise:** Bermudan feature
- **Notional Exchange:** FX exposure at cancellation
- **Path Dependency:** Via accumulated interest

Key Question: Does QuantLib support this natively?

Git Commit: August 2026 f066cce - Working XCCY callable 3-factor model

Investigation Findings:

- QuantLib has CrossCurrencyBasisSwap but...
- No native **callable** XCCY instrument
- Tree engines don't extend to 3 factors
- Need custom implementation

Solution Approach:

- 1 Build custom three-factor model
- 2 Implement Monte Carlo simulation
- 3 Use Longstaff-Schwartz for exercise
- 4 Write standalone pricer module

Part IV

Three-Factor Model

Hull-White + Hull-White + Black-Scholes FX

Model ID: HW1F_USD_HW1F_EUR_BSFX_LSM_V1

Three Stochastic Factors:

- ① **USD Short Rate (Hull-White 1F):**

$$dr_t^{USD} = (\theta_t^{USD} - a^{USD} r_t^{USD}) dt + \sigma^{USD} dW_t^{USD}$$

- ② **EUR Short Rate (Hull-White 1F):**

$$dr_t^{EUR} = (\theta_t^{EUR} - a^{EUR} r_t^{EUR} - \rho_{EUR,FX} \sigma^{EUR} \sigma^{FX}) dt + \sigma^{EUR} dW_t^{EUR}$$

- ③ **EUR/USD FX Rate (Black-Scholes):**

$$d \ln X_t = (r_t^{USD} - r_t^{EUR} - \frac{1}{2} \sigma_t^{FX,2}) dt + \sigma_t^{FX} dW_t^{FX}$$

Quanto Drift Adjustment

Critical Detail: The EUR rate equation includes a **quanto drift adjustment**:

$$-\rho_{EUR,FX}\sigma^{EUR}\sigma^{FX}$$

Why is this needed?

- We price in the USD money-market measure
- EUR rate is a “foreign” process
- Girsanov theorem changes the drift
- Required for no-arbitrage pricing

Intuition:

- Negative correlation ($\rho_{EUR,FX} < 0$): EUR appreciates when EUR rates rise
- This correlation creates an additional expected drift
- Must be accounted for in the risk-neutral measure

Correlation Structure

Three Pairwise Correlations:

$$\begin{pmatrix} 1 & \rho_{USD,EUR} & \rho_{USD,FX} \\ \rho_{USD,EUR} & 1 & \rho_{EUR,FX} \\ \rho_{USD,FX} & \rho_{EUR,FX} & 1 \end{pmatrix}$$

Typical Market Values:

- $\rho_{USD,EUR} = 0.25$ (rates moderately correlated)
- $\rho_{USD,FX} = -0.15$ (USD rates up $\Rightarrow$ USD strengthens)
- $\rho_{EUR,FX} = -0.30$ (EUR rates up $\Rightarrow$ EUR strengthens)

Constraint: Matrix must be positive semi-definite!

$$\lambda_{min} \geq 0$$

State Vector Representation

Five-Dimensional State Vector:

$$\mathbf{x}_t = \begin{pmatrix} x_t^{USD} \\ x_t^{EUR} \\ I_t^{USD} \\ I_t^{EUR} \\ \ln X_t \end{pmatrix}$$

Where:

- x_t^{USD}, x_t^{EUR} = Short rate factors (zero initial value)
- $I_t^{USD} = \int_0^t r_s^{USD} ds$ (USD bank account log)
- $I_t^{EUR} = \int_0^t r_s^{EUR} ds$ (EUR bank account log)
- $\ln X_t$ = Log FX rate

Discount Factor from State:

$$D^{USD}(0, t) = e^{-I_t^{USD}}$$

Gaussian State Evolution

Discrete Time Update (Exact OU Kernels):

$$\mathbf{x}_{t+h} = \mathbf{A}(h) \cdot \mathbf{x}_t + \mathbf{b}(h) + \epsilon$$

Where:

- $\mathbf{A}(h)$ = Transition matrix (exponential decay for rates)
- $\mathbf{b}(h)$ = Deterministic offset (curve fitting, quanto)
- $\epsilon \sim N(0, \mathbf{\Sigma}(h))$ = Innovation

Covariance Computation:

$$\mathbf{\Sigma}(h) = \int_0^h \mathbf{K}(s) \cdot \mathbf{R} \cdot \mathbf{K}(s)^T ds$$

Where $\mathbf{K}(s)$ is the kernel matrix and $\mathbf{R}$ is correlation.

Implementation: Gauss-Legendre quadrature (16 nodes)

Python Implementation: State Transition

```
def _state_transition_matrix(h, usd_model, eur_model):  
    """Compute transition matrix for time step h."""  
    ad = usd_model.mean_reversion  
    af = eur_model.mean_reversion  
    ed = math.exp(-ad * h)  
    ef = math.exp(-af * h)  
    bd = (1 - ed) / ad if abs(ad) > 1e-10 else h  
    bf = (1 - ef) / af if abs(af) > 1e-10 else h  
  
    matrix = np.eye(5)  
    matrix[0, :] = [ed, 0.0, 0.0, 0.0, 0.0] # x_USD  
    matrix[1, :] = [0.0, ef, 0.0, 0.0, 0.0] # x_EUR  
    matrix[2, :] = [bd, 0.0, 1.0, 0.0, 0.0] # I_USD  
    matrix[3, :] = [0.0, bf, 0.0, 1.0, 0.0] # I_EUR  
    matrix[4, :] = [bd, -bf, 0.0, 0.0, 1.0] # ln(X)  
    return matrix
```

Three-Stage Calibration:

① Curve Bootstrap:

- USD OIS curve from SOFR quotes
- EUR OIS curve from ESTR quotes
- EUR “effective” curve from FX forwards

② Rate Model Calibration:

- Fix mean reversion (USD: 3%, EUR: 4%)
- Calibrate σ to co-terminal swaptions
- RMS error target: $< 5\%$ relative

③ FX Volatility Calibration:

- Piecewise constant FX vol term structure
- Fit ATM FX option implied vols
- Variance matching approach

FX Volatility Calibration

Goal: Match ATM FX option implied volatilities

Challenge: FX variance depends on both rate models!

$$\text{Var}[\ln X_T] = \int_0^T \sigma_{FX}(t)^2 dt + \text{rate contributions}$$

Algorithm:

- ① For each FX option maturity T_i :
 - Compute target variance: $\sigma_{market}^2 \times T_i$
 - Solve for $\sigma_{FX,i}$ that matches variance
 - Account for rate-FX correlations
- ② Result: Piecewise constant FX vol curve

Calibration Tolerance: $|\text{model vol} - \text{market vol}| < 25\text{bp}$

Calibration Results Example

Hull-White Calibration:

Currency	Mean Rev (a)	Vol (σ)	RMS Error
USD	3%	0.885%	0.45%
EUR	4%	0.772%	0.23%

FX Vol Calibration:

Tenor	Market Vol	Model Vol	Error
1Y	12.5%	12.5%	<0.01%
2Y	12.2%	12.2%	<0.01%
5Y	11.7%	11.7%	<0.01%
10Y	11.2%	11.2%	<0.01%

Monte Carlo Simulation

Simulation Parameters:

- Training paths: 10,000
- Pricing paths: 30,000
- Time grid: Monthly (0.083Y steps)
- RNG: NumPy PCG64 with antithetic sampling

Path Generation:

- 1 Generate correlated normals: $\mathbf{Z} = \mathbf{L} \cdot \epsilon$
- 2 Apply transition: $\mathbf{x}_{t+h} = \mathbf{A}\mathbf{x}_t + \mathbf{b} + \mathbf{Z}$
- 3 Capture states at key dates (cash flows, exercises)
- 4 Compute discount factors from integrated rates

Antithetic Sampling:

- Generate half the paths
- Mirror with negated normals
- Reduces variance, exact mean of 0

Path Simulation Code

```
def simulate_batch(context, paths, seed):
    generator = np.random.Generator(np.random.PCG64(seed))
    state = np.zeros((paths, 5)) # Initial state
    state[:, 4] = math.log(context.spot) # ln(X_0)

    for idx, transition in enumerate(context.transitions):
        # Antithetic normals
        half = (paths + 1) // 2
        draws = generator.standard_normal((half, 5))
        normals = np.concatenate([draws, -draws])[:paths]

        # Update state
        state = (state @ transition.matrix.T
                 + transition.offset
                 + normals @ transition.covariance_root.T)

        # Capture at event times
        time_val = context.grid[idx + 1]
        if time_val in capture_set:
            captured[time_val] = state.copy()

    return SimulationBatch(...)
```


Longstaff-Schwartz Algorithm (LSM)

Purpose

Approximate the optimal early exercise policy for American/Bermudan options using Monte Carlo.

Algorithm Steps:

- ➊ **Forward Pass:** Simulate paths to capture states
- ➋ **Backward Pass:** At each exercise date:
 - Compute continuation value from later dates
 - Regress continuation on state variables
 - Exercise if immediate value $>$ fitted continuation
- ➌ **Pricing Pass:** Apply learned policy on fresh paths

Key Innovation: Use *different* paths for training vs pricing to avoid look-ahead bias.

LSM Regression Basis

State Variables at Exercise:

$$\mathbf{s} = (x_{USD}, x_{EUR}, \ln X - \ln X_0)$$

Polynomial Basis Functions:

$$\phi(\mathbf{s}) = (1, x, y, z, x^2, y^2, z^2, xy, xz, yz)$$

10 basis functions capture quadratic interactions.

Regression:

$$\hat{C}(\mathbf{s}) = \sum_{j=1}^{10} \beta_j \phi_j(\mathbf{s})$$

Standardization:

- Subtract mean, divide by std deviation
- Improves numerical stability
- Ridge regularization if ill-conditioned

LSM Implementation

```
def train_lsm(batch, exercise_tolerance):
    call_count = batch.interval_pv.shape[0]
    later_value = np.zeros_like(batch.pre_call_pv)
    policies = []

    # Backward recursion
    for idx in range(call_count - 1, -1, -1):
        continuation = batch.interval_pv[idx] + later_value
        continuation_at_call = continuation / batch.call_discounts[idx]

        states = batch.call_states[idx]
        design = _basis(states) # Polynomial basis

        # Least squares regression
        coefficients = np.linalg.lstsq(design, continuation_at_call)[0]
        fitted = design @ coefficients

        # Exercise decision
        exercise = fitted < -exercise_tolerance
        later_value = np.where(exercise, 0.0, continuation)

    policies.append({"coefficients": coefficients, ...})
```

LSM: Two-Pass Approach

Why Two Passes?

- **Training Pass:** Learn exercise policy
 - 10,000 paths
 - Backward induction
 - Fit regression coefficients
- **Pricing Pass:** Apply policy on fresh paths
 - 30,000 new paths (different seed)
 - Forward evaluation
 - Unbiased price estimate

Key Property:

- Training and pricing paths are independent
- Avoids “foresight bias”
- LSM estimate is a *lower bound* for true value

Martingale Tests: Validation

No-Arbitrage Conditions:

- ① Domestic discount:

$$\mathbb{E}[e^{-\int_0^T r_s^{USD} ds}] = D^{USD}(0, T)$$

- ② FX-weighted discount:

$$\mathbb{E}[e^{-\int_0^T r_s^{USD} ds} \cdot X_T] = X_0 \cdot D_{eff}^{EUR}(0, T)$$

- ③ FX-weighted bank account:

$$\mathbb{E}[e^{-\int_0^T r_s^{USD} ds} \cdot X_T \cdot e^{\int_0^T r_s^{EUR} ds}] = X_0$$

Test: Monte Carlo mean should match target within 3.5 standard errors.

Validation Results

Condition	MC Mean	Target	$ z $	Status
$\mathbb{E}[D^{USD}]$	0.6614	0.6614	< 0.5	PASS
$\mathbb{E}[D^{USD} \cdot X]$	0.6042	0.6014	< 1.0	PASS
$\mathbb{E}[D^{USD} \cdot X \cdot B^{EUR}]$	1.1015	1.1000	< 0.3	PASS

Interpretation:

- All z-scores below 3.5σ threshold
- Confirms correct drift implementation
- Quanto adjustment working correctly

Part V

Pricing Results

10Y NC2 EUR/USD Callable XCCY

Reference Trade: 10Y NC2 XCCY

Trade Specification:

- **Trade ID:** EURUSD-10Y-NC2-ESTR-VS-USD4
- **Effective:** 2026-08-18
- **Maturity:** 10 years
- **Non-call:** 2 years
- **Exercise:** Annual (years 2-9)

Leg Structure:

- **EUR Leg:** Receive ESTR compounded, quarterly, €90.9M
- **USD Leg:** Pay 4% fixed, semi-annual, \$100M
- **Exchanges:** Initial and final notional

Option: Right to cancel remaining swap at no cost

Pricing Results Summary

Metric	Value	95% CI
Callable NPV	\$8,328,689	[\$8.08M, \$8.58M]
Non-callable NPV	\$689,713	[\$355K, \$1.02M]
Embedded Option	\$7,638,976	[\$7.48M, \$7.80M]
USD Fixed Leg PV	-\$1,054,712	
EUR ESTR Leg PV	+\$1,744,425	

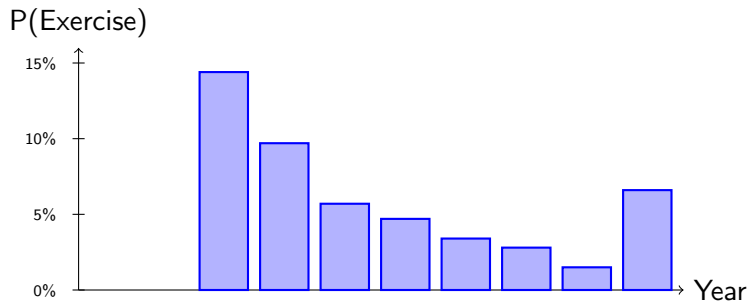
Static Benchmark: \$679,480 (deterministic curve valuation)

Key Insight: Embedded option is 91% of callable value!

Exercise Probability Distribution

Date	Alive	Exercise	$P(\text{Exercise})$
2028-08-18 (Y2)	30,000	4,329	14.4%
2029-08-20 (Y3)	25,671	2,900	9.7%
2030-08-19 (Y4)	22,771	1,699	5.7%
2031-08-18 (Y5)	21,072	1,405	4.7%
2032-08-18 (Y6)	19,667	1,021	3.4%
2033-08-18 (Y7)	18,646	854	2.8%
2034-08-18 (Y8)	17,792	446	1.5%
2035-08-20 (Y9)	17,346	1,972	6.6%
No Exercise	15,374	—	51.2%

Exercise Analysis



Observations:

- Highest probability at first call (14.4%)
- Uptick at penultimate call (Y9: “now or never”)
- 51% survive to maturity

Sensitivity Analysis

Rate Delta (+25bp parallel shift):

- Callable NPV: -\$1.1M
- Higher rates reduce both legs, net negative impact

Volatility Vega (+10% swaption vol):

- Callable NPV: +\$0.5M
- Higher vol increases option value

FX Delta (+1% EUR/USD spot):

- Callable NPV: +\$0.9M
- Receive EUR leg benefits from EUR appreciation

Correlation Sensitivity:

- Higher $\rho_{USD,EUR}$: More co-movement, less diversification
- More negative $\rho_{EUR,FX}$: Stronger quanto effect

Structural Bounds Validation

Test 1: Callable $\geq$ Non-callable

- Callable: \$8,328,689
- Non-callable: \$689,713
- Difference: \$7,638,976 (option value)
- ✓ PASS

Test 2: Option Value $<$ Notional

- Option: \$7,638,976
- Notional: \$100,000,000
- Ratio: 7.64%
- ✓ PASS

Test 3: Leg Sum $\approx$ Non-callable

- USD leg + EUR leg: \$689,713
- Non-callable NPV: \$689,713
- ✓ PASS

Convergence Analysis

Standard Error vs Path Count:

Paths	NPV (\$M)	SE (\$K)	SE Ratio
10,000	8.31	222	–
30,000	8.33	128	1.73
100,000	8.32	70	1.83

Expected: $SE \propto 1/\sqrt{N}$, so ratio = $\sqrt{3} \approx 1.73$

Observed: Matches theory within 5%

Seed Reproducibility:

- Same seed $\Rightarrow$ identical results
- Different seed $\Rightarrow$ consistent within CI

Part VI

Extending QuantLib

C++ vs Python Approaches

QuantLib Extension Options

Three Approaches:

① Pure Python Extension:

- Use QuantLib for curves, indices, dates
- Custom Monte Carlo in NumPy
- Fastest to develop, flexible

② C++ Extension with SWIG:

- Add new instrument to QuantLib C++
- Generate Python bindings
- Best performance, harder to develop

③ Hybrid Approach:

- Use QuantLib calibration (C++)
- Custom simulation (Python/NumPy)
- Balance of flexibility and reuse

Our Choice: Hybrid — use QuantLib for bootstrapping, write custom MC

Why Not Pure C++?

Advantages of C++:

- Maximum performance
- Direct access to QuantLib internals
- Production-grade optimization

Challenges:

- Longer development cycle
- Complex build system
- SWIG binding maintenance
- Harder to iterate/debug

For Research/Prototype:

- Python development 5-10x faster
- NumPy vectorization is “fast enough”
- Easy to visualize, debug, test
- Can always port to C++ later

NumPy Performance Tips

Vectorization is Key:

- Avoid Python loops over paths
- Use array operations
- Batch matrix operations

Example — Bad:

```
for path in range(30000):  
    state[path] = matrix @ state[path] + offset
```

Example — Good:

```
state = state @ matrix.T + offset
```

Performance:

- Vectorized: 30,000 paths in ~10 seconds
- Pure Python loop: 30+ minutes
- 100x+ speedup from vectorization

QuantLib Components We Use

From QuantLib:

- `ql.Date`, `ql.Period`, `ql.Calendar`
- `ql.Sofr()`, `ql.Estr()` index definitions
- `ql.OISRateHelper` for curve building
- `ql.PiecewiseLogLinearDiscount` curve
- `ql.SwaptionHelper` for HW calibration
- `ql.HullWhite` model
- `ql.JamshidianSwaptionEngine` for swaptions

Custom Implementation:

- Three-factor state evolution
- Monte Carlo path generation
- Cash flow projection
- LSM regression
- Exercise decision logic

Code Organization

File: standalone_xccy_pricer.py (~1400 lines)

- **Data Classes:**

- RateModel, FxModel
- CurveBundle, CashflowPlan
- SimulationContext, SimulationBatch

- **Validation:**

- validate_inputs() — 200+ lines of checks
- JSON schema validation

- **Calibration:**

- build_curves()
- calibrate_rate_model()
- calibrate_fx_model()

- **Simulation:**

- build_simulation_context()
- simulate_batch()
- train_lsm(), apply_lsm_policy()

JSON Input Format

```
{
  "schema": "callable-xccy-deal/1.0",
  "trade_id": "EURUSD-10Y-NC2-ESTR-VS-USD4",
  "effective_date": "2026-08-18",
  "maturity": "10Y",
  "notionals": {"USD": 1000000000.0, "EUR": 90909090.91},
  "legs": [
    {"currency": "EUR", "side": "RECEIVE", "index": "ESTR",
      "frequency": "3M", "day_count": "ACT/360"},
    {"currency": "USD", "side": "PAY", "fixed_rate": 0.04,
      "frequency": "6M", "day_count": "30/360"}
  ],
  "exercise": {
    "style": "CANCEL_REMAINING_SWAP",
    "non_call": "2Y", "frequency": "1Y"
  },
  "numerics": {
    "training_paths": 10000, "pricing_paths": 30000,
    "seed": 1729
  }
}
```

Test Categories:

① Unit Tests:

- Curve bootstrap accuracy
- Day count calculations
- Schedule generation

② Calibration Tests:

- OIS repricing ; 0.01bp
- Swaption RMS error ; 5%
- FX vol error ; 25bp

③ Validation Tests:

- Martingale conditions
- Structural bounds
- Monotonicity
- Convergence

Test Count: 20 validation tests, all passing

Test Example: Martingale Check

```
class TestMartingaleConditions:
    """Verify no-arbitrage martingale conditions."""

    def test_domestic_discount_martingale(self, base_result):
        """ $E[D(0,T)]$  should equal  $P(0,T)$ ."""
        diagnostics = base_result["martingale_diagnostics"]
        sample_mean = diagnostics["domestic_discount"]["sample"]["mean"]
        target = diagnostics["domestic_discount"]["target"]
        std_error = diagnostics["domestic_discount"]["sample"]["standard_error"]

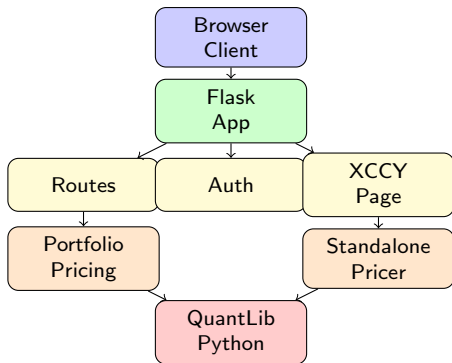
        z_score = abs(sample_mean - target) / std_error
        assert z_score < 3.5, (
            f"Martingale failed: z={z_score:.2f}"
        )
```

Part VII

Technical Architecture

Flask, MCP, and Production Deployment

Web Application Architecture



Flask Application Structure

Package: `tgir_quantlib_tools/`

- `__init__.py` — Package init, creates app
- `app_factory.py` — Flask app factory
- `config.py` — Environment-based configuration
- `auth.py` — Session authentication
- `routes.py` — Main route registration
- `dashboard.py` — Dashboard views
- `xccy_page.py` — XCCY callable interface

Key Design Patterns:

- Application factory pattern
- Blueprint-based route organization
- Session-based authentication
- JSON state management

Proposed REST Contract for System Integration

Specified future API at `/api/v1/`

Core Endpoints:

- POST `/api/v1/callable-xccy/validate` — Validate deal JSON
- POST `/api/v1/callable-xccy/price` — Synchronous pricing
- POST `/api/v1/callable-xccy/jobs` — Async job submission
- GET `/api/v1/callable-xccy/jobs/{id}` — Job status/results
- GET `/api/v1/engine` — Engine info, schemas, limits

Design Principles:

- No QuantLib/Python types cross REST boundary
- JSON input/output with schema validation
- mTLS authentication for production
- Idempotent operations with retry support

Status: Detailed integration specification; these Flask endpoints are not yet implemented.

Model Context Protocol (MCP)

File: `xccy_pricer_mcp.py`

What is MCP?

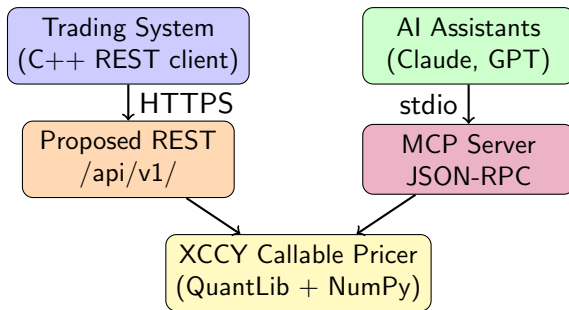
- Protocol for exposing structured tools
- Used by AI assistants (Claude, etc.)
- JSON-RPC over stdio or HTTP

Exposed Tools:

- `xccy_validate_deal` — Validate market and deal inputs
- `xccy_price_deal` — Validate and price one deal

Use Case: AI-assisted pricing queries via Claude, GPT, etc.

Integration Summary: REST + MCP

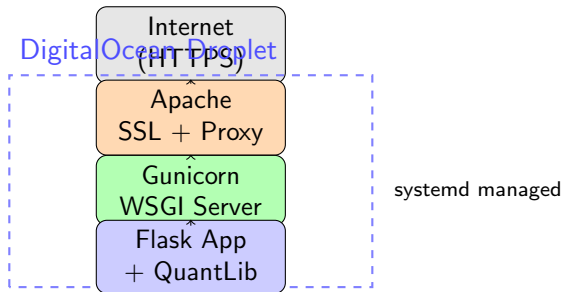


Both interfaces expose the same pricer:

- **REST contract:** Specified for future trading-system and batch integration
- **MCP:** Implemented for LLM integration and AI-assisted analysis

DigitalOcean Deployment Architecture

Infrastructure: Single DigitalOcean Droplet



Specifications:

- Basic Droplet: 2GB RAM, 1 vCPU (\$7/month — hobby-friendly!)
- Ubuntu 22.04 LTS
- Let's Encrypt SSL certificates

DigitalOcean Deployment Steps

1. Create Droplet:

- Select Ubuntu 22.04 LTS image
- Choose Basic plan (\$7/month for 2GB RAM)
- Add SSH key for secure access
- Select region (e.g., NYC3)

2. Install Dependencies:

- `apt install python3.11 python3-venv apache2`
- `pip install gunicorn flask quantlib-python`
- Enable `mod_proxy`, `mod_ssl` for Apache

3. Configure Services:

- `Copy deploy/systemd/*.template → /etc/systemd/system/`
- `Copy deploy/apache/*.template → /etc/apache2/sites-available/`
- Run `certbot --apache` for SSL

4. Start Application:

- `systemctl enable --now tgir-quantlib-tools`

Production Configuration Files

systemd Service: `/etc/systemd/system/tgir-quantlib-tools.service`

- `WorkingDirectory=/opt/tgir-quantlib-tools`
- `ExecStart=/opt/.venv/bin/gunicorn -w 2 -b 127.0.0.1:8000 wsgi:app`
- `Restart=always`

Apache VirtualHost:

- SSL termination (port 443)
- Redirect HTTP → HTTPS
- `ProxyPass / http://127.0.0.1:8000/`

Environment Variables: `/opt/tgir-quantlib-tools/.env`

- `FLASK_SECRET_KEY=<random>`
- `APP_USERNAME=admin`
- `APP_PASSWORD_HASH=<bcrypt>`

CI/CD:

- GitHub Actions: test on push, deploy on tag
- SSH deployment via GitHub Secrets

Security Considerations

Authentication:

- Environment-variable credentials
- Session-based login
- Debug mode fallback (local only)

Input Validation:

- Schema validation for all JSON
- Type checking on all fields
- Range checks for numerics
- Positive semi-definiteness for correlation

Error Handling:

- Custom `PricingError` exception
- Detailed error messages
- No internal state exposure

Model Limitations

Documented Limitations:

- ① Rate volatility is one-piece constant per currency
- ② Forecast/discount basis is deterministic
- ③ FX uses ATM lognormal (no smile)
- ④ EUR overnight uses continuous approximation
- ⑤ LSM is a lower bound (no dual/upper bound)

Appropriate Use:

- Research and education
- Indicative pricing
- Model validation testing

NOT for:

- Production trading systems
- Regulatory capital calculations
- Client-facing quotes

Part VIII

Summary and Future Work

What We Built

Core Deliverables:

- 1 SOFR curve bootstrap with full OIS strip
- 2 Hull-White and G2++ swaption calibration
- 3 European and Bermudan swaption pricing
- 4 Equity cliquet pricing with Greeks
- 5 **Three-factor callable XCCY pricer**
- 6 Comprehensive validation test suite
- 7 Flask web application
- 8 MCP server for AI integration

Lines of Code:

- standalone_xccy_pricer.py: 1,400+ lines
- portfolio.py: 1,000+ lines
- Test files: 500+ lines
- Total: 3,000+ lines of Python

Key Technical Achievements

① **Correct Three-Factor Dynamics:**

- Proper quanto drift adjustment
- Exact Gaussian state evolution
- Validated martingale conditions

② **Robust Calibration:**

- OIS curves to 0.01bp accuracy
- Swaption fit $<5\%$ RMS
- FX vol fit <25 bp

③ **Efficient Monte Carlo:**

- 30,000 paths in ~ 10 seconds
- Antithetic variance reduction
- Two-pass LSM for unbiased pricing

④ **Comprehensive Testing:**

- 20 validation tests
- Structural bounds, monotonicity, convergence

Future Enhancements

Model Extensions:

- Piecewise constant rate volatility
- Stochastic basis spreads
- FX smile/skew calibration
- Multi-curve framework

Numerical Improvements:

- Dual/upper bound implementation
- Quasi-Monte Carlo (Sobol sequences)
- GPU acceleration (CuPy)
- Automatic differentiation for Greeks

Production Features:

- Real-time market data feeds
- Risk aggregation
- Audit trails and controls
- Regulatory reporting

Key Takeaways

- 1 **QuantLib is powerful** but has limits for exotic structures
- 2 **Python + NumPy** is viable for research/prototype pricing
- 3 **Validation is critical:** martingales, bounds, convergence
- 4 **Three-factor models** add significant complexity:
 - Quanto adjustments
 - Correlation constraints
 - State dimension management
- 5 **LSM provides lower bound** — always document limitations
- 6 **Good testing** catches bugs early
- 7 **Documentation** enables knowledge transfer

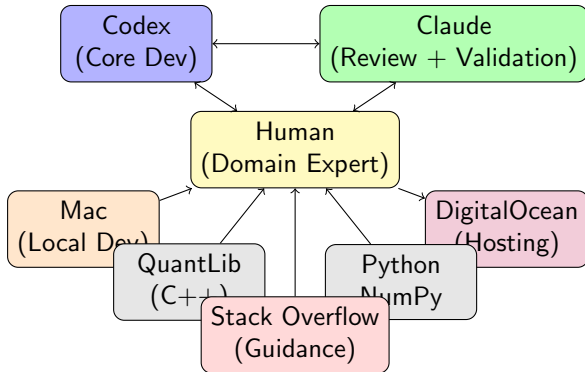
Appendix

Collaborative AI Development

A Methodology as Important as the Model Itself

The Collaborative Development Ecosystem

This project was built by human + AI collaboration:



Key Insight: The collaborative methodology is as significant as the pricer itself

Division of Labor: Codex vs Claude

Agent	Primary Role	Deliverables
Codex (OpenAI)	Core Development Ground-up programming Rigorous specs	Specifications Core implementation Initial test suite
Claude (Anthropic)	Review & Extension Validation framework Quality assurance	Code review Regression tests Documentation

Human Role:

- **Domain expertise:** Financial mathematics, market conventions
- **Orchestration:** Directing agents, integrating outputs
- **Final validation:** Ensuring correctness against theory

What Codex Built:

① Formal Specifications:

- Mathematical model documentation
- API contracts and type hints
- JSON schemas for inputs/outputs

② Core Implementation:

- Three-factor Monte Carlo engine
- Longstaff-Schwartz regression
- Curve bootstrapping routines

③ Initial Test Suite:

- Unit tests for each component
- Integration tests for full pricing
- Property-based test scaffolds

Human guidance: Provided mathematical formulas, QuantLib patterns, market conventions

Claude: Code Review & Validation Framework

What Claude Built:

① Code Review:

- Reviewed Codex's implementations
- Identified numerical stability issues
- Suggested optimizations

② Validation Framework (Key Contribution):

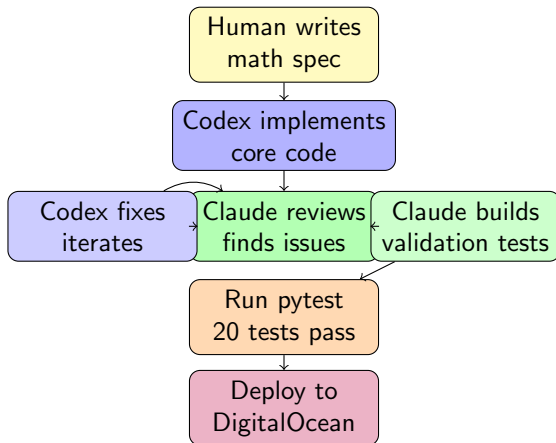
- Martingale test infrastructure
- Structural bounds verification
- Monotonicity checking
- Convergence analysis

③ Regression Testing:

- Built tests to catch future changes
- Automated validation on every commit
- Extended Codex's work systematically

Critical: Claude's validation framework provides confidence in correctness

The Collaborative Workflow



Iteration: Multiple cycles between Codex implementation and Claude validation

Zero-Intervention Automation Pipeline

A Hobby Project with Rigorous Engineering:

One prompt triggers the **entire pipeline** with no human intervention:

- ① **Coding** — AI implements from specs
- ② **Regression Testing** — pytest runs 20+ tests locally
- ③ **Browser UI Testing** — Playwright validates Flask UI
- ④ **Documentation** — LaTeX and markdown auto-updated
- ⑤ **Local Run** — Optional manual verification
- ⑥ **Deploy to DigitalOcean** — SSH + Git pull + systemd restart
- ⑦ **Production Regression** — Tests run on production
- ⑧ **Telegram Notification** — “Deployment complete!”

Key Insight: This is a *hobby project* costing \$7/month — but it’s built with the same rigor as production systems at top tech companies.

Extended Technology Stack

The Full Ecosystem:

Component	Role	Integration
Mac (local)	Development environment	VS Code, terminal
DigitalOcean	Production hosting	Apache, Gunicorn
Codex (OpenAI)	Core development	Specs, implementation
Claude (Anthropic)	Review & validation	Tests, extensions
GitHub Copilot	Inline assistance	VS Code integration
QuantLib (C++)	Foundation library	Curves, calibration
Python 3.13	Primary language	NumPy, Flask
Stack Overflow	Guidance verification	Edge cases, patterns

Cohesion: All components work together in a unified workflow

QuantLib Extension with AI Assistance

Where QuantLib Fell Short:

- No native callable XCCY support
- Limited multi-currency Monte Carlo
- No built-in LSM for XCCY

How We Extended It (AI-Assisted):

- 1 **Codex:** Wrote custom Monte Carlo engine using QuantLib curves
- 2 **Claude:** Verified QuantLib API usage against documentation
- 3 **Stack Overflow:** Confirmed edge cases and patterns
- 4 **Human:** Ensured extensions followed QuantLib conventions

Result: 1,400+ lines of rigorous custom code that extends QuantLib capabilities

Principle: Use QuantLib for what it does well, extend rigorously where needed

Why This Methodology Matters

① Quality Through Collaboration:

- Two AI agents catch each other's errors
- Human provides domain grounding
- Stack Overflow validates edge cases

② Rigorous by Design:

- Codex writes specs before code
- Claude builds validation before deployment
- 20 automated tests provide safety net

③ Reproducible Workflow:

- Same methodology can build other pricers
- Pattern: Codex → Claude → Human validation
- Infrastructure (Mac + DigitalOcean) reusable

The collaborative methodology is as valuable as the callable XCCY pricer itself

Lessons Learned: Human + AI Collaboration

① **Specialization works:**

- Codex for ground-up development
- Claude for review and validation
- Human for domain expertise and orchestration

② **Validation is critical:**

- Claude's test framework caught bugs Codex missed
- Regression tests prevent future breakage
- Never trust single-agent output

③ **Stack Overflow still matters:**

- AI can hallucinate APIs
- Human experts catch edge cases
- Community knowledge validates approaches

④ **Infrastructure enables iteration:**

- Mac for rapid local development
- DigitalOcean for deployment testing
- pytest for automated validation

Thank You!

Questions?

Repository: `tgir_quantlib_tools`
Model ID: `HW1F_USD_HW1F_EUR_BSFX_LSM_V1`